GO PROGRAMMING MASTER COURSE

Week 10, Day 1

Goroutines and the scheduler — concurrency is not a free lunch in HTTP handlers.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST WEEK

Errors mapped. Logs structured. Still one request, one stack.

- Missing student → 404 via `ErrNotFound`. No panics in handlers.
- Cold-call: what is a goroutine? (guesses OK — we name it today.)
- Today we start concurrent work — and see a race before we “optimize” SMS.
- Channels / worker pools wait until week 11. Mutex deep-dive week 12.

TODAY

If Day 1 of week 10 works, you can

- Say concurrency vs parallelism in one sentence each.
- Start work with `go f()` and explain it is not a thread API.
- Sketch GMP (Goroutine / Machine / Processor) at poster level.
- Spot a leaking goroutine and why handlers must not fire-and-forget.
- Run `go test -race` on a broken counter and see red.

SOFTWARE ENGINEERING

Concurrency is a product decision

SMS create does not need a goroutine. Reach for concurrency when latency or fan-out earns it — after you can prove safety.

- Async log-after-create is tempting — and races the map (week 12).
- Prefer a clear sync path today; add concurrent validation as a kata.
- Never go work in a handler with no cancel, no wait, no error path.

CONCURRENCY VS PARALLELISM

Two words. Do not mix them.

Concurrency

Many tasks in progress — structure. One CPU can still interleave them.

Parallelism

Many tasks at the same instant — needs multiple cores / Ps.

- Go gives you concurrency tools. Parallelism is what the scheduler may do.

GOROUTINES

go f() — cheap concurrent function call

```
go say("world")
say("hello") // may pri

func say(s string) {
    for i := 0; i < 3;
        fmt.Println(s,
    }
}
```

- Not “new OS thread” for each go — multiplexed onto a few threads.
- Returns immediately; you do not wait unless you arrange it.

SCHEDULER · GMP POSTER

Enough picture for interviews and fear

- **G** — goroutine (your `go f()`).
- **M** — OS thread (machine).
- **P** — logical processor (scheduling context; ~GOMAXPROCS).
- Runnable Gs wait on queues; Ms run Gs using Ps.
- Do not memorize source — remember: many Gs, few Ms.

LEAKS

A goroutine that never ends still costs

```
go func() {
    for {
        // blocked forever on a
    }
}()
// request finished — G still a
```

- Leaks grow memory and scheduler work under load.
- Week 11/12: channels + context cancel as exit ramps.

HTTP HANDLERS

Do not go without a plan

```
func (h *StudentHandler) Create(c *gi
    // BAD: go h.repo.Create(req)  //
    s := h.repo.Create(req)       //
    c.JSON(201, s)
}
```

- Client needs the outcome on this request.
- Background work needs: ownership, cancellation, and error reporting.

DATA RACE · PREVIEW

Two goroutines, one counter, no lock

```
var n int
var wg sync.WaitGroup
for i := 0; i < 1000; i+
    wg.Add(1)
    go func() { defer wg
}
wg.Wait()
fmt.Println(n) // often
```

- `go test -race` will scream. Fix with mutex is **week 12**.
- Today: feel the bug; do not ship “async create” into SMS yet.

GO TEST -RACE

The detector is your friend

```
go test -race ./..
# WARNING: DATA RA
# Write at  ...  Rea
```

- Lab starts red on purpose.
- Homework: optional race in an unlocked global map — document it; fix next week.

SAFER LAB

Fan-out three validations

```
// email / course / phone check
// collect errors, then respond
// discuss: is this overkill fo
```

- Independent CPU/IO-ish checks can fan out.
- SMS fields are tiny — sync validation is usually enough.
- Learn the pattern; do not force it into every handler.

SMS MILESTONE

Practical, not theatre

- Keep create/list sync and correct.
- Optional later: timeout-friendly slow path / dashboard aggregation.
- Or: async log *only after* you have seen the race you will fix in week 12.
- No channels API design this week.

LIVE DEMO

10K GOROUTINES

Printing is fine · shared counter is not

```
for i := 0; i < 10_000; i++ {
    go func(i int) { fmt.Println(i)
}
// vs incrementing one int from many
```

LIVE DEMO

- RACE

Red is a lesson

```
cd goroutine-lab
go test -race ./...
# DATA RACE on counter_test.
```

Leave it red in class. Mutex arrives week 12.

LIVE DEMO

FAN-OUT

Three checks, one result

```
errs := ValidateStudentConcurrent(req)
// non-nil errs → 400 with joined messages
// compare timing vs ValidateStudentSync (optio
```

ANTI-PATTERN

Naked go in Create

- Client gets 201 before work finishes — or never learns failure.
- Map writes race with List/Get.
- Process exit can kill work mid-flight.
- Course rule this week: **no naked go in handlers**.

LAB · REST OF CLASS

Done when all of this is true

- Ran 10k-goroutine print demo (or equivalent)
- `go test -race` shows DATA RACE on the counter lab
- Fan-out validation runs; can explain sync vs concurrent tradeoff
- SMS create path still sync; health green
- Quiz answers started for homework

HOMEWORK

Quiz + optional race note

- Short quiz: concurrency vs parallelism; what `go f()` does; why handlers must not naked-go.
- Optional: if your in-memory store used a global map without a mutex under concurrent tests, capture `-race` output in the PR.
- Do not “fix” the race yet unless you already know mutex — week 12.

NEXT SESSION

Week 11 — channels, pipelines, worker pool

- Unbuffered vs buffered; `select`; range/close.
- Worker pool as the pattern to remember.
- Bring race literacy and respect for naked go.

WORDS TO KEEP

Say these correctly next week

Goroutine

Concurrent function scheduled by Go.

GMP

G=goroutine, M=thread, P=processor.

Data race

Concurrent write/read without sync.

Leak

Goroutine that never exits.

LAYOUT REMINDER

Where lab code lives

```
docs/week-10-day-01/starter/
  goroutine-lab/          # race counter + fan-out val
  backend/                # SMS still sync create/list
    cmd/server/main.go
    internal/ ...
```

HONEST LIMITS

What this week is not

- Not channels / worker pools (week 11).
- Not mutex mastery (week 12) — we only observe races.
- Not “make SMS faster with go” theatre.
- Not GOMAXPROCS tuning class.

SAFETY

Still true under concurrency demos

- Do not commit secrets while experimenting.
- Cap demo loops — 10k prints is enough; avoid locking the room projector.
- `gofmt`; no binaries in git.

REVIEW BAR

Instructor merges when

- Quiz answers show clear concurrency vs parallelism
- No naked `go` merged into Create without discussion
- Race lab evidence (`-race` output) if they claim a race
- SMS health + create still green

Questions